Splunk-SIEM-Lab-v2 Production Grade Review

Repository review and how to ship the fixes

Emmanuel Tigoue

May 19, 2026

Splunk-SIEM-Lab-v2 Production Grade Review

Repository: github.com/umujalloh/Splunk-SIEM-Lab-v2 **Reviewer:** Emmanuel Tigoue **Date:** May 19, 2026

Verdict

Top decile student work. Better than most working SOC analysts produce in their first two years. Structure, framework mapping, and honest documentation of dead ends signal senior level thinking.

The notes below are the difference between strong student work and production grade detection engineering. The fixes are about packaging the reasoning for production reviewers, not changing the reasoning itself.

What You Nailed

1. **Hypothesis driven structure.** Investigative question before every query, then SPL, then findings. Most students dump queries with no reasoning.
2. **Dead end documentation.** The `splunk.froth.ly` false lead and the pivot to `PerfmonMk` is in the README, not hidden. Single most senior move in the repo.
3. **Two host comparison.** BSTOLL-L mined for 26 minutes, BTUN-L blocked 46 attempts. Same vector, different outcome, framed as a detection coverage gap. Detection engineering thinking, not just IR.
4. **Honest epistemics on T1071.001.** WebSocket inferred from activity pattern, not directly observed. That confidence labeling passes senior interviews.
5. **Visibility gap callouts.** Explicit notes when data is missing. Turning a gap into a recommendation instead of stopping at "I cannot tell."
6. **Framework mapping density.** Four frameworks (MITRE ATT&CK, NIST CSF, 800-53, CIS Controls v8) all tied to specific queries. Not slapped on.
7. **IOCs.md as a real artifact.** Hashes, signature IDs (30356, 30358), sub-IDs, behavioral patterns, three packaged detection queries. Useful to a downstream analyst.

Production Grade Gaps

1. Query Traceability and Double Counting

`PerfmonMk:Process` samples fire every second per process. "131 events at 99 to 100% CPU sustained over 26 minutes" might be double counting the same `chrome.exe` instance, or counting child Chrome processes separately. In production this miscounts severity and triggers wrong escalation.

Fix. Add deduplication or instance level aggregation.

```
index=botsv3 sourcetype="PerfmonMk:Process"
host="BSTOLL-L" instance=*chrome*
%_Processor_Time>=99
| dedup _time, instance
| stats min(_time) as first_seen,
        max(_time) as last_seen,
        count by instance
```

Re-state the finding with the dedup count as a range (e.g. "X distinct sample windows across Y chrome process instances over 26 minutes").

2. No False Positive Profile on the CPU Rule

Your proposed rule:

```
sourcetype="PerfmonMk:Process" instance="*chrome*" %_Processor_Time>=90
| stats count by host, instance
| where count > 100
```

Will fire on every Zoom screenshare, Webex meeting, video encode, Adobe Lightroom export, Chrome with 15 tabs, Slack call, Teams call.

Fix. Pair the rule with four controls.

- **Process allowlist.** Exempt known high CPU processes during business hours.
- **Multi signal requirement.** CPU spike AND DNS to a known mining or C2 indicator within 10 minutes. CPU alone is not enough.
- **Baseline window.** Compute the user's normal Chrome CPU over a 14 day window. Alert only when current usage exceeds baseline plus N standard deviations.
- **Time of day weighting.** 3am 100% CPU is more suspicious than 2pm.

Add a "False Positive Analysis" section to the README listing these for each detection rule.

3. Least Privilege Analysis Missing

Critical for the osTicket build. The agent will hold three credentials at minimum. Each needs a documented scope **before code**.

Credential	Minimum Scope
Splunk service account	Read only on specific indexes (`index=tickets`, `index=auth`). No write. No admin. No deploy.
osTicket admin API key	Comment and priority update only. No delete. No ticket creation. No user management.
Claude API key	Daily token budget cap. Rate limit (e.g. 60 RPM). Audit log every call.

Containerize the agent with no network egress except to allowlisted endpoints (Splunk URL, osTicket URL, api.anthropic.com). All credentials rotatable, all audit logged on use.

4. No Detections as Code

Your three detection SPL queries live at the bottom of IOCs.md. In production they live in a `detections/` folder, one file per rule, version controlled, with unit tests against synthetic events.

```
repo/
  detections/
    coinhive-dns.spl
    chrome-sustained-cpu.spl
    jscoinminer-symantec-baseline.spl
    README.md
  tests/
    test_coinhive_dns.py
    test_chrome_cpu.py
```

Hiring managers grep for `detections/` when reviewing portfolios.

5. No D3FEND Mapping

You mapped MITRE ATT&CK (attacker side) but not MITRE D3FEND (defender side). D3FEND is the defensive counterpart and is increasingly required for federal and DFARS work, with growing adoption in commercial.

Fix. Add a "Defender Mapping (D3FEND)" subsection under MITRE.

ATT&CK	D3FEND Countermeasure
T1189 Drive-by Compromise	D3-WHL Web Header Logging, D3-NTA Network Traffic Analysis
T1059.007 JavaScript	D3-PSEP Process Self-Modification Detection
T1071.001 Application Layer Protocol: Web	D3-DNSAL DNS Allowlisting, D3-DNSDL DNS Denylisting
T1496 Resource Hijacking	D3-PR Process Restriction, D3-RAPA Resource Access Pattern Analysis

6. No CVE Traceback

`brewertalk.com` was compromised through outdated forum software (likely MyBB or vBulletin). Without identifying the CVE, the detection does not feed vulnerability management.

Fix. Research the CVE behind the 2018 brewertalk compromise and add a "Root Cause" section tying exploit chain to vulnerability.

7. No Business Impact Statement

"26 minutes of mining" is technical. A CISO reads:

- **Direct cost.** Estimated kilowatt hours of electricity translated to operational cost.
- **Productivity impact.** 26 minutes of degraded performance times affected users.
- **Compliance impact.** PCI DSS 5.1 (anti-malware), HIPAA 164.308(a)(6) (security incident procedures), SOX implications if BSTOLL-L touched financial data.
- **Reputational risk.** Notification obligations if customer data was processed on the affected host.

Add a "Business Impact" section. This is how analysts communicate up the chain.

8. No Purple Team Closing Loop

You wrote detection rules but did not prove they fire. A production detection engineer writes the rule, then writes a test that simulates the attack and validates the rule catches it.

Fix. Use Atomic Red Team or Caldera to simulate JSCoinminer behavior. Document that the test fires the detection. Without a closing loop, the detection is unproven.

9. No Threat Intel Currency Check

CoinHive infrastructure was sunsetted in March 2019. The exact IOCs you found are no longer relevant. A production analyst notes this and proposes current Monero pool patterns.

Fix. Add a "Threat Intel Update" section. Reference current cryptomining campaigns (XMRig, 8220 gang, Outlaw, TeamTNT). Propose updated detection patterns for 2026 threats.

10. Detection Latency Analysis Missed

Symantec blocked BTUN-L at 09:37:40. BSTOLL-L started mining 10 seconds later at 09:37:50. Defenders had a 10 second early warning signal on an adjacent host that did nothing to protect BSTOLL-L.

Fix. Propose a correlation rule that, when one EDR signal fires, automatically elevates monitoring on adjacent hosts within the same subnet for the next 60 minutes. The data is in the dataset. You did not pull on the thread.

11. No Loom or YouTube Walkthrough

Hiring managers do not finish READMEs. A 5 to 7 minute video walking the hunt end to end (screen recording in Splunk, narrating the hypothesis pivots) would 10x the impact of this work.

Fix. Record a Loom. Link it at the top of the README under "Walkthrough Video."

Where This Work Lives

The work in v2 is core threat hunting and detection engineering. Companies doing this:

- **MSSPs:** Binary Defense, Arctic Wolf, Expel, GuidePoint, eSentire, Trustwave
- **In-house SOCs:** Huntington Bank, Nationwide, Target, large healthcare and finance
- **Threat hunting specialists:** Mandiant, CrowdStrike Falcon OverWatch, Red Canary

The osTicket triage agent maps directly to the hottest space in security right now. Tines, Torq, Dropzone AI, Anvilogic, and Hunters are all building SOAR plus LLM products. Polishing v2 and shipping the osTicket build is the difference between landing a Tier 1 SOC seat and landing a detection engineering or security automation role.

Recommended Claude Code Skills

Install these into your Claude Code setup. They cover the gaps the fixes above will surface.

Skill	What it gives you	Source
owasp-security	OWASP Top 10:2025, ASVS 5.0, **LLM Top 10 (2025)**, and **Agentic AI security (2026)**. The LLM and Agentic AI sections are exactly what you need for the osTicket agent threat model.	Claude Code skills marketplace
cybersecurity-pack	ATT&CK Navigator layers, threat hunting workflows, detection content patterns. Built for security work in Claude Code.	github.com/mukul975/Anthropic-Cybersecurity-Skills
grc-pack	Per-framework skills (NIST CSF, PCI DSS, HIPAA, ISO 27001, SOC 2, FedRAMP, ISO 42001 AI). Use for your framework mapping and business impact section.	github.com/Sushegaad/Claude-Skills-Governance-Risk-and-Compliance
GSD (Get Shit Done)	Project execution methodology described below.	github.com/glittercowboy/get-shit-done

For owasp-security specifically: when you sit down to write the seven attack threat model for the osTicket agent, load that skill first. The Agentic AI 2026 section covers prompt injection, confused deputy, and indirect exfil with current language and current mitigations. Saves you 4 hours of research.

GSD (Get Shit Done)

GSD is a set of Claude Code commands that wrap your work in atomic commits, phase planning, verification gates, and wave-based execution. The output is a clean git history with one atomic commit per task and an audit trail of decisions a hiring manager can scan.

- **GitHub:** https://github.com/glittercowboy/get-shit-done
- **Install:** `npx get-shit-done-cc` inside Claude Code
- **Discord:** https://discord.gg/5JJgD5svVS

Commands relevant to this build:

Command	When to use
`/gsd:new-project`	Starting the osTicket build. Generates a structured roadmap before code.
`/gsd:plan-phase`	Planning a phase with task breakdown and verification loop.
`/gsd:execute-phase`	Executing the plan with wave-based parallelization. Multiple agents on independent tasks.
`/gsd:quick`	Small atomic tasks (the v2 polish fixes). Skips heavy planning, keeps atomic commits.
`/gsd:progress`	Check where you are and route to next action.
`/gsd:debug`	Systematic debugging with state preserved across sessions.
`/gsd:resume-work`	Resume a previous session with full context restored.
`/gsd:add-todo`	Capture an idea or task without breaking your current flow.

How to apply it:

1. For the v2 polish wave, use `/gsd:quick` per fix. Each fix is a short atomic task.
2. For the osTicket architecture doc, use `/gsd:plan-phase`. The threat model is the deliverable.
3. For the osTicket build, run `/gsd:new-project` first to scaffold the roadmap. Then `/gsd:plan-phase` per sub-phase. Then `/gsd:execute-phase` to ship.
4. When you hit a bug you cannot solve in 15 minutes, hit `/gsd:debug`. The investigation persists across sessions.

Path Forward

Four waves. Run them in order. Use the GSD commands above to manage each.

Wave 1: v2 Polish

Touch only the existing repo. No new code.

- Add `/queries` folder, one .spl file per query
- Add a False Positive Analysis section under each detection rule
- Add D3FEND mapping subsection under the existing MITRE mapping
- Add a CVE root cause section
- Add a Business Impact section
- Add a Threat Intel currency update note (CoinHive sunsetted 2019, list current threats)
- Re-run the "131 events" finding with `| dedup` and re-state with the corrected count
- Record a 5 to 7 minute Loom walkthrough, link it at the top of the README

Scope: Small. Use `/gsd:quick` per item.

Wave 2: osTicket Architecture Doc

Ship as a PR to a new repo **before any code**. The doc is the interview artifact.

- One page components and data flow diagram
- Seven attack threat model (below)
- Phasing plan (read only first, write second)
- Least privilege spec for all three credentials
- Idempotency strategy (ticket ID tracking, webhook retry safety)
- Observability plan (audit logs to Splunk)
- Kill switch design (one env var disables all writes)

Seven attacks to threat model:

1. Prompt injection from ticket bodies
2. Confused deputy (agent modifying tickets it was not asked about)
3. Indirect data exfil via internal notes
4. False negative on severity (real incident marked low)
5. False positive on severity (helpdesk ticket pages analyst at 3am)
6. API key compromise
7. Replay or duplicate processing

Scope: Medium. Use `/gsd:plan-phase`. Load the owasp-security skill first.

Wave 3: Build osTicket Phase 1 through 3

Use `/gsd:new-project` to scaffold the roadmap, then `/gsd:plan-phase` per sub-phase, then `/gsd:execute-phase`.

- **Phase 1.** Webhook receiver (FastAPI) plus Claude classifier with forced JSON schema output. No writes.
- **Phase 2.** Add Splunk enrichment, read only, named indexes. Still no writes.
- **Phase 3.** osTicket internal note updates only. Audit log every classification to Splunk from day one.

Scope: Largest. Use `/gsd:execute-phase`.

Wave 4: Public Artifacts

- Three LinkedIn posts: v2 polish writeup, threat model walkthrough, agent demo
- One YouTube or Loom: full end to end demo with narration
- Public GitHub repo with the agent code, sanitized

Scope: Small. Use `/gsd:quick` per artifact.

Bottom Line

The bones of this work are senior level already. The eleven fixes package the reasoning for production reviewers. Every one can land in a focused session with `/gsd:quick`. After that, the osTicket build is the differentiator that takes you from “I documented a hunt” to “I built detections and the agent that triages them.”